

// THE CLUSTER

```
image      apache/kafka:3.8.1 (KRaft, 1 broker)
in-cluster kafka:9092
from laptop localhost:29092 (Java client)
cli in ctr  localhost:9092
kafka-ui    http://localhost:8081
```

Serialization is **JSON only** in this course — no Schema Registry, no Avro.

// COMMAND PATTERN

Scripts aren't on PATH — use the full path inside the container.

```
docker compose -f infra/compose.yaml exec kafka \
/opt/kafka/bin/kafka-topics.sh \
--bootstrap-server localhost:9092 --list
```

Below, `kbin` = `/opt/kafka/bin`, all with `--bootstrap-server localhost:9092`.

// TOPICS

```
kbin/kafka-topics.sh --list

kbin/kafka-topics.sh --create \
--topic applicant-registered \
--partitions 3 --replication-factor 1

kbin/kafka-topics.sh --describe \
--topic applicant-registered
# read Leader / Replicas / Isr (all 1)
```

// PRODUCE (KEYED)

```
kbin/kafka-console-producer.sh \
--topic applicant-registered \
--property parse.key=true \
--property key.separator=:
# then type:
1001:{"name":"Ada","country":"GB"}
1001:{"name":"Ada","country":"GB"}
```

Same key → same partition. That's how ordering per applicant is guaranteed.

// CONSUME

```
kbin/kafka-console-consumer.sh \
--topic applicant-registered \
--from-beginning \
--property print.partition=true \
--property print.key=true
```

The log is not a queue — re-reading from the beginning replays everything.

// CONSUMER GROUPS

```
# consume as a group member
kbin/kafka-console-consumer.sh \
--topic applicant-registered --group screening

# inspect lag
kbin/kafka-consumer-groups.sh \
--describe --group screening
# CURRENT-OFFSET · LOG-END-OFFSET · LAG
```

// RESET OFFSETS

```
kbin/kafka-consumer-groups.sh --group screening \
--topic applicant-registered \
--reset-offsets --to-earliest --dry-run
# swap --dry-run for --execute to apply
# --to-datetime for point-in-time
```

// LIVE CONFIG

```
kbin/kafka-configs.sh --alter \
--entity-type topics --entity-name applicant-registered \
--add-config retention.ms=600000
```

```
kbin/kafka-configs.sh ... \
--add-config cleanup.policy=compact
# compaction keeps the last message per key
```

// REBALANCING

Add or kill a consumer in group `workers` — partitions are reassigned live across the surviving members.

// JAVA CLIENTS (HOST)

```
# examples/kafka-clients
ProducerMain / ConsumerMain
# connect via host port 29092
```

Pointing a host client at `9092` hangs — that's the internal listener, not the host one.